Software Architecture Document

Delivery 2

Moreno Barragán Jhonatan David
Gaitán Castañeda Luna Valeria
Ochoa Silva Jonathan Felipe
Ortiz Luna Nicolás Francisco

October, 10, 2026

Contents

1 Executive Summary **5**

2 System Context **6**
2.1 System Overview 6
2.2 External Actors and Systems 6
2.3 System Context Diagram 8
2.4 External Interactions 9

3 Software Architecture **11**
3.1 Architectural Approach 11
3.2 System Decomposition 11
3.3 Services and Main Responsibilities 11
3.4 Architecture Diagram 12
3.5 Architectural Principles 13

4 Service Catalog **14**
4.1 SVC-01 – Account and Driver Management 14
4.1.1 Responsibility 14
4.1.2 APIs 14
4.1.3 Data Owned 15
4.1.4 Dependencies 15
4.1.5 Scalability 15
4.2 SVC-02 – Ride Request Management 16

4.2.1 Responsibility . . . 16
4.2.2 APIs . . . 16
4.2.3 Data Owned . . . 16
4.2.4 Dependencies . . . 16
4.2.5 Scalability . . . 16
4.3 SVC-03 – Matching and Dispatch . . . 17
4.3.1 Responsibility . . . 17
4.3.2 APIs . . . 17
4.3.3 Data Owned . . . 17
4.3.4 Dependencies . . . 17
4.3.5 Scalability . . . 17
4.4 SVC-04 – Trip Management and Tracking . . . 18
4.4.1 Responsibility . . . 18
4.4.2 APIs . . . 18
4.4.3 Data Owned . . . 18
4.4.4 Dependencies . . . 19
4.4.5 Scalability . . . 19
4.5 SVC-05 – Pricing and Geospatial . . . 19
4.5.1 Responsibility . . . 19
4.5.2 APIs . . . 19
4.5.3 Data Owned . . . 20
4.5.4 Dependencies . . . 20
4.5.5 Scalability . . . 20
4.6 SVC-06 – Payment and Settlement . . . 20
4.6.1 Responsibility . . . 20
4.6.2 APIs . . . 20
4.6.3 Data Owned . . . 21
4.6.4 Dependencies . . . 21
4.6.5 Scalability . . . 21
4.7 SVC-07 – Feedback and Safety . . . 21
4.7.1 Responsibility . . . 21
4.7.2 APIs . . . 22
4.7.3 Data Owned . . . 22
4.7.4 Dependencies . . . 22
4.7.5 Scalability . . . 22
4.8 SVC-08 – Notifications and Communications . . . 23
4.8.1 Responsibility . . . 23
4.8.2 APIs . . . 23
4.8.3 Data Owned . . . 23

4.8.4 Dependencies . . . 23
4.8.5 Scalability . . . 23

5 Data Architecture **24**
5.1 Data Storage Strategy . . . 24
5.2 Data Models . . . 25
5.3 Data Ownership . . . 25
5.4 Consistency . . . 26
5.5 Transactions . . . 26
5.6 Data Architecture Diagram . . . 27

6 Communication Architecture **27**
6.1 Communication Strategy . . . 27
6.2 Synchronous Communication . . . 27
6.3 Asynchronous Communication . . . 28
6.4 Communication Diagram . . . 29
6.5 Communication Justification . . . 29

7 Architecture Decision Records **30**
7.1 ADR-001 – Service Decomposition . . . 30
7.1.1 Context . . . 30
7.1.2 Decision . . . 30
7.1.3 Alternatives . . . 31
7.1.4 Rationale . . . 31
7.1.5 Trade-offs and Consequences . . . 31
7.2 ADR-002 – Service Data Ownership . . . 31
7.2.1 Context . . . 31
7.2.2 Decision . . . 31
7.2.3 Alternatives . . . 31
7.2.4 Rationale . . . 32
7.2.5 Trade-offs and Consequences . . . 32
7.3 ADR-003 – Hybrid Communication Strategy . . . 32
7.3.1 Context . . . 32
7.3.2 Decision . . . 32
7.3.3 Alternatives . . . 32
7.3.4 Rationale . . . 32
7.3.5 Trade-offs and Consequences . . . 33
7.4 ADR-004 – Consistency and Transaction Management . . . 33
7.4.1 Context . . . 33
7.4.2 Decision . . . 33

7.4.3 Alternatives . . . 33
7.4.4 Rationale . . . 34
7.4.5 Trade-offs and Consequences . . . 34
7.5 ADR-005 – Independent Scalability and Geographic Distribution . . . 34
7.5.1 Context . . . 34
7.5.2 Decision . . . 34
7.5.3 Alternatives . . . 34
7.5.4 Rationale . . . 35
7.5.5 Trade-offs and Consequences . . . 35

8 Requirements Traceability **35**
8.1 Functional Requirements Traceability . . . 35
8.2 Non-Functional Requirements Traceability . . . 36
8.3 Architecture Decision Traceability . . . 37
8.4 Traceability Summary . . . 37

1 Executive Summary

This document defines the software architecture of the global ride-sharing platform specified in Delivery 1, where the team characterized the business problem, identified all actors and use cases, stated the functional and non-functional requirements, and produced a quantitative workload model with up to 500,000 concurrent users, 120 million requests per hour at peak, 99.99% annual availability, and a p95 response time below 500 ms for latency-sensitive operations. The purpose of this delivery is to translate those requirements into a concrete, cloud-independent software design: a set of services, the data each service owns, the communication patterns that connect them, and the architectural decisions that justify each choice.

Architectural approach. The platform follows a **service-oriented, event-driven architecture** composed of eight independently deployable services organized into four groups by business capability: the marketplace core (SVC-02 to SVC-05), money and trust (SVC-06, SVC-07), identity and onboarding (SVC-01), and communications (SVC-08). Every external client reaches the platform through a single **API Gateway**, which centralizes routing, authentication, and rate limiting. Services interact synchronously (REST/gRPC) only when an immediate response is required on the critical path, such as fare and ETA quoting. The rest of the traffic (driver telemetry, trip lifecycle transitions, payment side effects, and notifications) flows asynchronously through an **Event Bus** with at-least-once delivery, so that no secondary workflow blocks matching or trip tracking. Each service owns its data store exclusively and the storage technology is chosen per service rather than shared across the platform.

Design objectives. The architecture pursues four objectives aligned with the requirements and workload of Delivery 1. First, it must sustain the 45 to 120 million requests per hour of the workload while keeping the rider's critical path under 500 ms. Second, it must isolate failure domains so that a problem in payments, feedback, or notifications never degrades matching or trip tracking. Third, every piece of data has a single writer: cross-service access is handled through APIs or event subscriptions rather than shared storage. Fourth, the design remains technology-agnostic, so the mapping to concrete infrastructure can be produced in Delivery 3 without changing the service model. These objectives are realized through the nine architectural principles stated in the Software Architecture section (single responsibility per service, exclusive data ownership, right communication for each interaction, stateless by default, design for failure, independent scaling and regional deployment, security by design, observability, and cloud independence).

Relation to Delivery 1. Every architectural element of this document traces back to a requirement established in Delivery 1, as detailed in the Requirements Traceability section. The service decomposition (ADR-001) covers the full actor catalog and the ten functional requirements FR-01 to FR-10. Independent scaling and geographic distribu-

tion (ADR-005) address the workload and availability requirements NFR-01 to NFR-04, NFR-06, and the global scope of NFR-09. The hybrid communication strategy (ADR-003) responds to the mixed latency and throughput profile of NFR-02, NFR-03, and NFR-05: the rider's critical path remains synchronous and bounded, while high-volume streams (driver telemetry alone accounts for roughly 60% of the workload) travel through the Event Bus. The data ownership and transaction model (ADR-002, ADR-004) honours the dual correctness/availability profile identified in Delivery 1, keeping high-volume telemetry eventually consistent while payments, driver ledger, and trip-state transitions remain strongly consistent and auditable. No architectural element appears without a requirement to justify it, in line with the traceability discipline this delivery demands.

2 System Context

2.1 System Overview

At the highest level the system is a single, logical *Ride-Sharing Platform* that operates a two-sided marketplace: riders request transportation and drivers provide it, and the platform coordinates every step of the lifecycle from driver verification and quoting through matching, trip tracking, fare computation, settlement orchestration, feedback, and safety. Seen from the outside, the platform is a black box reachable through two client-facing surfaces (the rider and driver mobile applications) and two operator-facing surfaces (the Support/Safety console and the Market Administration console). All ingress is terminated by the API Gateway, which is a single entry point from the perspective of this context view.

Internally, the platform owns driver verification and onboarding workflows, supply and demand matching and dispatch, the trip lifecycle and its telemetry, in-house routing and geospatial logic built on top of external map data, dynamic pricing and fare computation, settlement orchestration, notification orchestration, and all operational tooling required to keep each market healthy. It does **not** own the physical transportation, the devices, the payment network, the identity bureaus, the map data provider, or the message-delivery carriers. Those sit outside the boundary and define the external interfaces specified in this section.

The actors and systems listed below are the complete set identified in Delivery 1 and are carried over without modification, so that the C4 Level 1 view remains consistent with the actor and use case catalog of the previous delivery.

2.2 External Actors and Systems

The platform interacts with four human actor types and four external systems. Table 1 states, for each one, what it is, the nature of its interaction with the platform, and the

expected frequency.

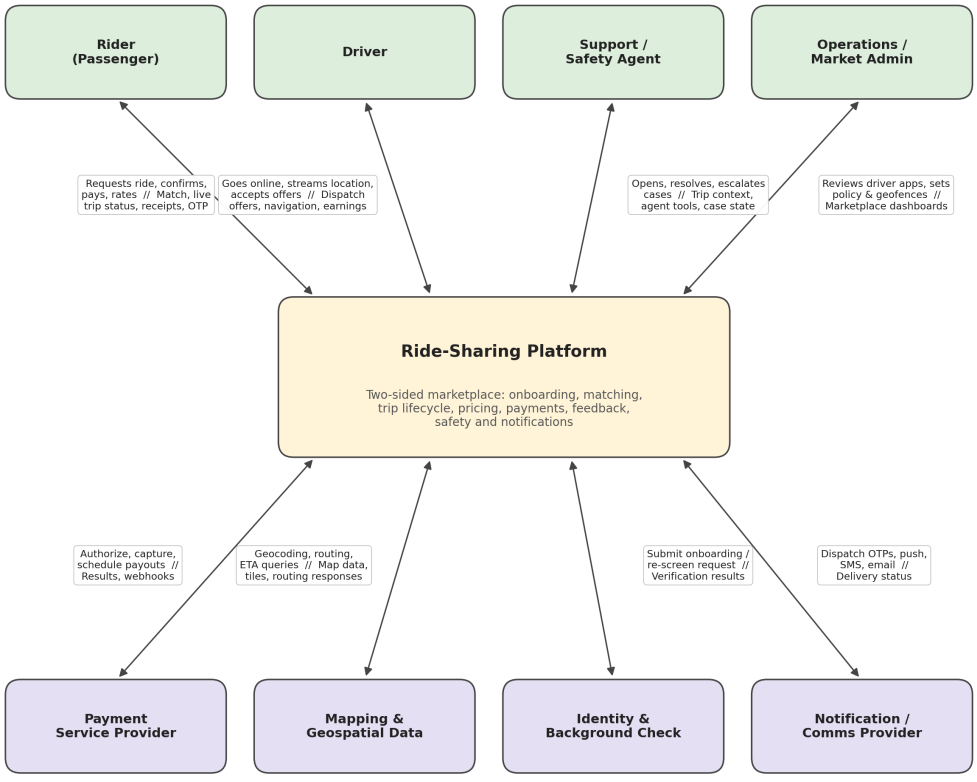
External element	Nature	Frequency	Description
Rider (Passenger)	Human (primary)	Occasional, active per trip	End user who requests transportation; expects fast matching, an up-front price, a trackable trip and support when something goes wrong.
Driver	Human (primary)	Continuous while online	Independent driver who offers availability, accepts dispatch offers, carries riders, and is paid. Streams location continuously while online.
Support / Safety Agent	Human (secondary)	Event-driven, 24/7	Internal operator who handles support cases, disputes, and safety escalations. Interacts with the platform only when a case is open.
Operations / Market Administrator	Human (secondary)	Periodic, administrative	Internal operator responsible for a city or region; reviews driver onboarding, sets pricing policy and service areas, and watches marketplace health.
Payment Service Provider	System (secondary)	Per trip (authorize, capture, payouts)	External payment processor that authorizes rider charges, captures them at trip end, and schedules driver payouts; it never stores raw card data inside the platform.
Mapping & Geospatial Data Provider	System (secondary)	Continuous, high volume	External source of base maps, geocoding, and routing data used by the platform's own ETA, pricing, and trip-tracking logic.
Identity & Background Verification Provider	System (secondary)	Onboarding and periodic re-screening	External provider that verifies driver identity and runs background checks at onboarding and on the re-check schedule.

External element	Nature	Frequency	Description
Notification / Communications Provider	System (secondary)	Event-driven, frequent	External carriers that actually deliver OTPs, push notifications, SMS, and email on behalf of the platform.

Table 1: External actors and systems that interact with the Ride-Sharing Platform.

2.3 System Context Diagram

Figure 1 shows the platform as a single black box at C4 Level 1, surrounded by the four human actor types and the four external systems listed above. The bi-directional arrows summarise both directions of each interaction and are labelled with the nature of the information crossing the boundary. The internal decomposition of the platform (services, data stores, event bus) is intentionally hidden at this level and is developed in the Software Architecture section.



C4 Level 1. Bi-directional arrows summarise both directions of the interaction (sent // received). Communication patterns (sync vs. async) are described in the Software Architecture and Communication Architecture sections.

Figure 1: System Context Diagram (C4 Level 1) of the ride-sharing platform.

2.4 External Interactions

Table 2 describes each crossing of the system boundary: the direction, the information exchanged, and the expected frequency, derived from the workload of Delivery 1 (45 million requests per hour on average, up to 120 million at peak, with driver telemetry accounting for roughly 60% of the volume).

External party	**Direction**	**Interaction**	**Expected frequency**
Rider	→ Platform	Submits ride request, confirms the quote, cancels before pickup, pays, rates the trip, opens a support case or safety report.	Several calls per ride; globally, hundreds of thousands of active ride sessions at peak.
Rider	← Platform	Receives fare and ETA quote, match confirmation, live driver location and trip status, receipts, OTPs, and notifications.	Continuous during an active trip (streaming).
Driver	→ Platform	Goes online/offline, streams location updates, accepts or rejects dispatch offers, confirms pickup and completion.	Continuous telemetry stream plus discrete actions per trip; dominant share of overall traffic.
Driver	← Platform	Receives dispatch offers, navigation guidance, trip details, earnings updates, and notifications.	Continuous while online.
Support / Safety Agent	↔ Platform	Opens, inspects, and resolves support tickets and safety incidents; reads trip context on demand; triggers refunds or account actions through platform services.	Only when a case is open; 24/7 availability required.
Operations / Market Administrator	↔ Platform	Reviews driver applications, defines pricing rules and service areas, watches marketplace dashboards.	Periodic, administrative; business hours per market.

External party	Direction	Interaction	Expected frequency
Payment Service Provider	↔ Platform	The platform requests authorization at the start of a trip, capture at trip completion, and payouts on schedule; the provider returns results synchronously and may also send asynchronous webhooks for settlement events.	Typically two synchronous calls per completed trip (authorize + capture) plus payout batches.
Mapping & Geospatial Data Provider	→ Provider	The platform calls the provider for base maps, geocoding, and routing data used to produce ETAs and track trips. Responses are cached per geographic cell to limit calls.	Continuous and high volume: roughly one call per quote and one per telemetry batch.
Identity & Background Verification Provider	↔ Platform	The platform submits onboarding and re-screening requests; the provider returns authoritative, auditable verification results, which may also arrive asynchronously.	Low volume: on onboarding and on the re-check schedule per driver.
Notification / Communications Provider	→ Provider	The platform dispatches OTPs, push notifications, SMS, and email on behalf of users; the provider reports delivery status back through webhooks.	Several messages per trip; peaks together with the trip workload.

Table 2: Interactions between the platform and its external actors and systems.

The interactions above are consistent with the actor catalog and the use cases defined in Delivery 1. UC-01 (*Request and match a ride*) and UC-02 (*Accept and perform a trip*) drive the interactions among Rider, Driver, and the Mapping provider; UC-03 (*Complete trip and process payment*) drives the Payment Service Provider interaction; UC-04 (*Onboard and verify a driver*) drives the Identity & Background Verification Provider interaction; and UC-05 (*Rate and report a trip*) drives the Support / Safety Agent interaction. All four external systems are consumed on the platform's side of the boundary, which is why the software architecture presented in the next section assigns each one to a specific internal service: the Mapping provider to SVC-05 (Pricing and Geospatial), the Payment provider to SVC-06 (Payment and Settlement), the Identity & Background

provider to SVC-01 (Account and Driver Management), and the Notification provider to SVC-08 (Notifications and Communications).

3 Software Architecture

3.1 Architectural Approach

The platform follows a **service-oriented, event-driven architecture** composed of eight independently deployable services organized around the business capabilities (bounded contexts) identified in Delivery 1. External clients (rider and driver applications, support and administration consoles) reach the platform through a single **API Gateway**. Services interact in two ways:

- **Synchronously** (REST/gRPC), when the caller needs an immediate answer on the critical path, such as obtaining a fare estimate.
- **Asynchronously** through an **Event Bus** (at-least-once delivery), for high-frequency streams and for workflows that must not block each other, such as driver telemetry or trip completion.

Each service exclusively owns its data store (polyglot persistence, see Section 5). The design is cloud-independent: every component is described by its role (gateway, message broker, relational or document store) and not by a specific product.

3.2 System Decomposition

The decomposition separates capabilities with different workload profiles and consistency needs, as established in Delivery 1.

Two cross-cutting platform components complement the business services: the **API Gateway** (routing, token validation, rate limiting, TLS termination) and the **Event Bus** (durable, partitioned topics with at-least-once delivery).

3.3 Services and Main Responsibilities

Boundary justifications.

- SVC-02 and SVC-04 are separate because a ride request has no driver and a short life, whereas a trip is long-lived and write-heavy.
- SVC-03 is separate from SVC-04 because matching needs in-memory, geographically sharded state, while trips need durable history.
- Payments are isolated from trips so that a payment failure never blocks trip completion.

Group	Services	Why grouped
Marketplace core (real-time, latency-sensitive)	SVC-02, SVC-03, SVC-04, SVC-05	Directly determine the p95 $<$ 500 ms target (NFR-05). They carry most of the 45M–120M requests/hour (NFR-02, NFR-03), mainly driver telemetry.
Money and trust (correctness-sensitive)	SVC-06, SVC-07	Require strong consistency, audit trails and idempotency. They are not on the rider's latency-critical path.
Identity and onboarding	SVC-01	Low-volume, high-integrity data (credentials, verification status).
Communications	SVC-08	Pure event consumer. Isolates delivery failures from business flows.

Table 3: Service groups and rationale

ID	Service	Main responsibility
SVC-01	Account and Driver Management	Accounts, authentication, driver onboarding and verification status.
SVC-02	Ride Request Management	Lifecycle of a ride request before a driver is assigned (quote, confirm, cancel, expire).
SVC-03	Matching and Dispatch	Driver availability, candidate search, dispatch offers and assignment.
SVC-04	Trip Management and Tracking	Trip state machine after assignment; real-time telemetry ingestion and tracking.
SVC-05	Pricing and Geospatial	Fare estimates and final fares, surge per geographic cell, geofences and service areas.
SVC-06	Payment and Settlement	Payment methods, charges, cash reconciliation, driver ledger, payouts and invoices.
SVC-07	Feedback and Safety	Ratings, support tickets and safety incidents.
SVC-08	Notifications and Communications	Decides, renders and dispatches OTPs, push, SMS and email.

Table 4: Services and their main responsibility

3.4 Architecture Diagram

Figure 2 shows the services, their groupings and the interaction topology. Solid arrows are synchronous calls and dashed arrows are asynchronous events exchanged through the Event Bus.

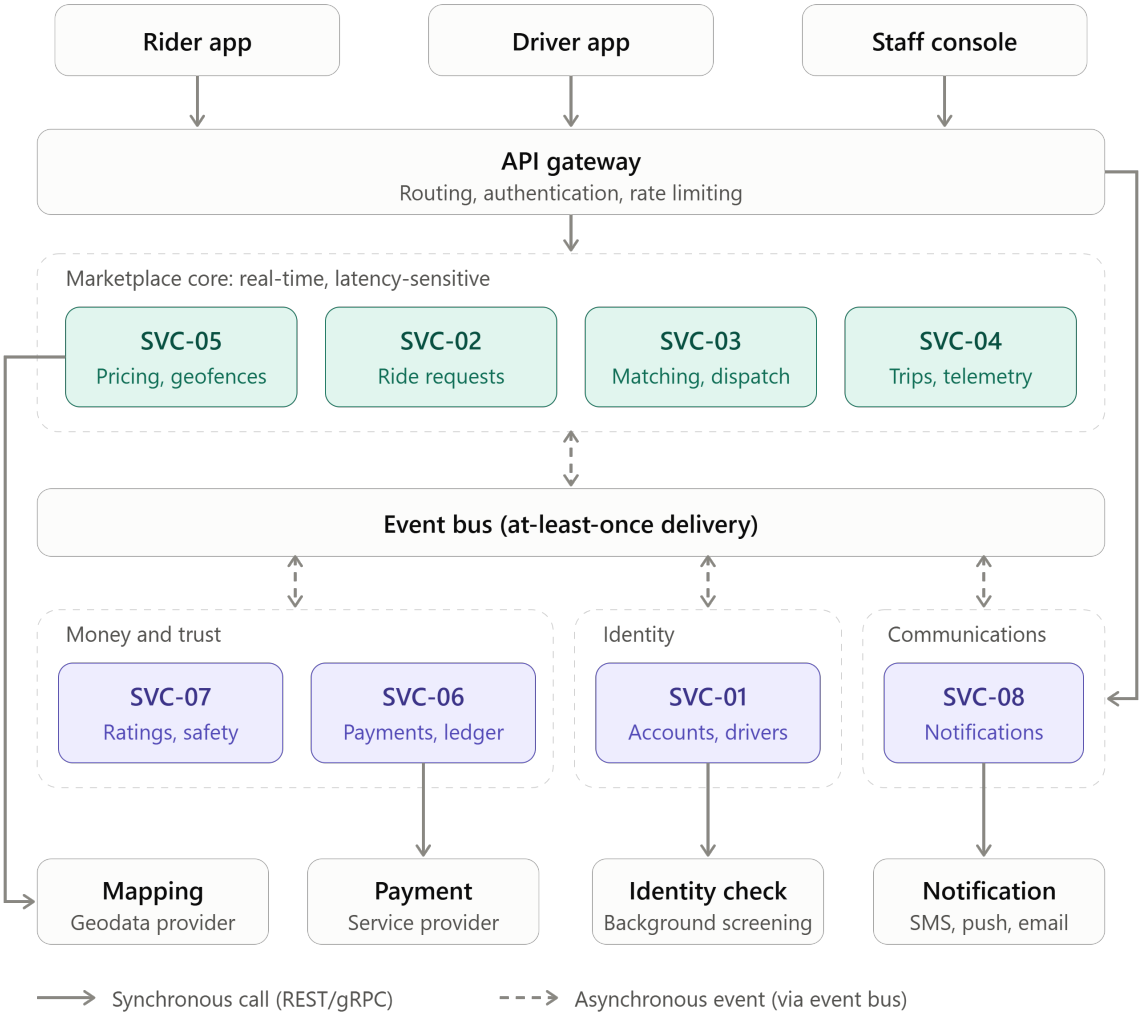


Figure 2: Software architecture of the ride-sharing platform

3.5 Architectural Principles

1. **Single responsibility per service.** Each service answers one business question and nothing else.

2. **Exclusive data ownership.** Only the owner writes its data. Other services use APIs or event subscriptions (no shared databases).

3. **Right communication for each interaction.** Synchronous only on the critical path; asynchronous for decoupling, high volume or non-blocking workflows.

4. **Stateless by default.** Services are stateless and horizontally scalable where possible; state lives in the data tier. Stateful services (SVC-03) are partitioned by geographic cell.

5. **Design for failure.** Idempotent consumers, idempotency keys, timeouts, retries

with backoff, circuit breakers, and sagas with compensation instead of distributed transactions.

6. **Independent scaling and regional deployment.** Each service scales on its own driver (requests, queue depth, memory) and can be deployed per region.

7. **Security by design.** Central token validation at the gateway, least-privilege service access, and tokenized payment data only.

8. **Observability.** Correlation identifiers on every request and event, plus structured logs and metrics per service.

9. **Cloud independence and deployability.** Services are containerizable and configured through the environment, so they can be mapped to infrastructure in Delivery 3.

4 Service Catalog

Conventions. All paths are versioned (`/v1`). Authentication is validated at the API Gateway. Elements in curly braces, such as `{userId}`, are *path parameters*: placeholders replaced by a real identifier at call time (e.g., `/v1/users/8f3a21`). Paths are relative to the base address of each service, not to any file system. "Internal" endpoints are reachable only from other services. Events are published to the Event Bus with at-least-once delivery, so every consumer is idempotent. Instance counts are global estimates derived from the Delivery 1 workload (about 12,500 requests/s on average and 33,300 at peak), assuming roughly 60% of traffic is driver telemetry. Every service keeps at least two instances per region for availability. The estimates will be validated in Delivery 3.

4.1 SVC-01 – Account and Driver Management

4.1.1 Responsibility

Manages rider and driver accounts, authentication, profiles and the driver onboarding workflow (document submission, verification, approval). It issues access tokens and exposes driver eligibility. It does **not** manage driver online/offline availability (SVC-03), ratings (SVC-07) or payment data (SVC-06). *Traceability: FR-01, UC-04.*

4.1.2 APIs

Method	Path	Description
POST	`/v1/auth/register`	Create a rider or driver account.
POST	`/v1/auth/login`	Authenticate (credentials plus OTP) and issue tokens.
POST	`/v1/auth/refresh`	Renew an access token.
GET / PATCH	`/v1/users/{userId}`	Read or update the profile.
POST	`/v1/drivers/applications`	Submit an onboarding application (identity, license, vehicle).
POST	`/v1/drivers/applications/{id}/documents`	Upload or resubmit documents.
GET	`/v1/drivers/applications/{id}`	Check onboarding status.
PUT	`/v1/admin/drivers/applications/{id}/decision`	Administrator approves, rejects or holds.
GET	`/v1/internal/drivers/{driverId}/eligibility`	Internal: is the driver active and verified?

Events published: `DriverApproved`, `DriverSuspended`, `UserProfileUpdated`. **Events consumed:** `SafetyIncidentReported` (serious cases, may trigger suspension).

4.1.3 Data Owned

`User`, `DriverProfile`, `Vehicle`, `DocumentVerification` (relational database). Credentials are stored hashed. Identity documents are kept in object storage and referenced from the database.

4.1.4 Dependencies

- **Identity & Background Verification Provider** (external): identity validation and screening, including periodic re-screening.
- **SVC-08** (synchronous): sends OTP codes.
- **Event Bus**: publishes driver status changes and consumes safety events.

4.1.5 Scalability

Stateless. Scales on request volume (login bursts at commute peaks). Estimate: **3 instances on average / 6 at peak**. Onboarding workflows are asynchronous per application, so the latency of the external provider does not block the service.

4.2 SVC-02 – Ride Request Management

4.2.1 Responsibility

Owns the ride request from creation to hand-off: validates input, obtains the fare and ETA quote, lets the rider confirm or cancel, and expires stale requests. It does **not** search for drivers (SVC-03), compute prices (SVC-05) or manage the trip once a driver is assigned (SVC-04). *Traceability: FR-03, FR-05, UC-01.*

4.2.2 APIs

Method	Path	Description
POST	`/v1/ride-requests`	Create a request (pickup, destination, service type). Returns a quote (fare and ETA) obtained from SVC-05.
GET	`/v1/ride-requests/{id}`	Current status of the request.
POST	`/v1/ride-requests/{id}/confirm`	Rider confirms the quote and `RideRequested` is published. Reconfirmation is required if surge changed.
POST	`/v1/ride-requests/{id}/cancel`	Rider cancels before pickup.

Events published: `RideRequested`, `RideRequestCancelled`, `RideRequestExpired`.
Events consumed: `RideMatched`, `NoDriverFound` (update request status).

4.2.3 Data Owned

`RideRequest` (pickup, destination, service type, quote snapshot, status, timestamps) in a document store.

4.2.4 Dependencies

- **SVC-05** (synchronous), `GET/v1/estimates`: fare and ETA, latency target < 100 ms.
- **Event Bus**: publishes request events and consumes matching results.

4.2.5 Scalability

Stateless. Scales on request volume. Estimate: **4 instances on average / 10 at peak**. Confirmation is idempotent through an idempotency key, which makes client retries safe.

4.3 SVC-03 – Matching and Dispatch

4.3.1 Responsibility

Tracks which drivers are available, finds candidate drivers near a request using a hexagonal spatial index, issues dispatch offers, handles accept, reject and timeout, and assigns the driver. It does **not** store trips (SVC-04) or compute fares (SVC-05). *Traceability: FR-02, FR-04, UC-01, UC-02.*

4.3.2 APIs

Method	Path	Description
PUT	`/v1/drivers/{driverId}/availability`	Driver goes online or offline.
GET	`/v1/drivers/{driverId}/offers/current`	Fetch the pending offer (fallback to push).
POST	`/v1/offers/{offerId}/accept`	Driver accepts the offer.
POST	`/v1/offers/{offerId}/reject`	Driver rejects the offer.

Events published: `RideMatched`, `NoDriverFound`, `DispatchOfferCreated`, `DriverAvailabilityCha`
Events consumed: `RideRequested`, `RideRequestCancelled`, `DriverLocationUpdated`, `DriverApproved`, `DriverSuspended`, `TripCompleted`, `TripCancelled` (the last two return the driver to the available pool).

4.3.3 Data Owned

`ActiveDriverSession` (availability, last known cell), `DispatchOffer`, `RiderDriverMatch`, in an in-memory data grid / key-value store. This is operational state that can be rebuilt from the event stream.

4.3.4 Dependencies

- **Event Bus**: main input (requests and locations) and output (matches).
- **SVC-01**: driver eligibility, kept as a local copy fed by `DriverApproved` and `DriverSuspended` events (synchronous call only as fallback).
- **SVC-05** (optional, synchronous): service-area lookup.

4.3.5 Scalability

Stateful, partitioned by geographic cell, so each partition handles one city or area. Scales on event throughput and memory (active driver sessions). Estimate: **8 instances on**

average / 24 at peak. Re-partitioning follows demand concentration. If a partition is lost, it is rebuilt from recent location events.

4.4 SVC-04 – Trip Management and Tracking

4.4.1 Responsibility

Creates the trip when a match occurs and manages its state machine (en route to pickup, in progress, completed, cancelled). It ingests driver telemetry, serves live location to riders and stores route history. It requests the final fare at completion and publishes `TripCompleted`. It does **not** process payments (SVC-06) or decide matches (SVC-03). *Traceability: FR-06, FR-07, UC-02.*

4.4.2 APIs

Method	Path	Description
POST	`/v1/trips/{tripId}/pickup-confirmation`	Driver confirms pickup (trip starts).
POST	`/v1/trips/{tripId}/complete`	Driver ends the trip.
POST	`/v1/trips/{tripId}/cancel`	Cancel (rider no-show, interruption).
PATCH	`/v1/trips/{tripId}/destination`	Mid-trip change of stop or destination.
GET	`/v1/trips/{tripId}`	Trip state and details.
POST	`/v1/trips/{tripId}/locations`	Driver sends a location update (or equivalent streaming channel).
GET	`/v1/trips/{tripId}/location`	Latest location.
WS	`/v1/trips/{tripId}/stream`	Real-time progress stream for the rider.
GET	`/v1/trips/{tripId}/route`	Route history.

Events published: `DriverLocationUpdated`, `TripStarted`, `TripCompleted`, `TripCancelled`, `TripStatusChanged`. **Events consumed:** `RideMatched` (creates the trip).

4.4.3 Data Owned

`TripState`, `TelemetryPing`, `RouteHistory` in a wide-column / time-series store. `TripState` transitions are strongly consistent; telemetry is eventually consistent.

4.4.4 Dependencies

- **SVC-05** (synchronous), `POST/v1/fares/final`: final fare at completion, included in the `TripCompleted` payload.
- **Event Bus**: consumes matches and publishes telemetry and trip lifecycle events.

4.4.5 Scalability

Stateless processing; real-time rider streams are held per connection and recover by reconnecting. It is the **highest-volume service**: about 60% of all requests (around 7,500 pings/s on average and 20,000/s at peak). Scales on write throughput and open connections. Estimate: **10 instances on average / 28 at peak**. Telemetry is batched and the topic is partitioned by geographic cell.

4.5 SVC-05 – Pricing and Geospatial

4.5.1 Responsibility

Calculates fare and ETA estimates and final fares, maintains surge multipliers per geographic cell from supply and demand signals, and manages geofences, service areas and per-market pricing rules. It does **not** hold trips or requests. *Traceability: FR-05, FR-07, UC-01, UC-03.*

4.5.2 APIs

Method	Path	Description
GET	`/v1/estimates`	Fare and ETA by service tier (query: pickup, destination, service type).
POST	`/v1/fares/final`	Internal: final fare for a completed trip (distance, time, surge, fees).
GET	`/v1/surge`	Current surge multiplier for a cell or location.
GET	`/v1/geofences/resolve`	Resolve market and service area for a coordinate.
PUT	`/v1/admin/pricing-rules/{ruleId}`	Administrator sets pricing policy per market.
POST / PUT	`/v1/admin/geofences`	Administrator manages service areas.

Events published: `SurgeMultiplierUpdated`. **Events consumed:** `RideRequested` (demand), `DriverAvailabilityChanged` (supply).

4.5.3 Data Owned

`Geofence`, `PricingRule`, `SurgeMultiplier` in a relational database with spatial extensions. Current surge values are also served from a cache for low latency.

4.5.4 Dependencies

- **Mapping & Geospatial Data Provider** (external): base maps, geocoding and routing data for ETA.
- **Event Bus**: demand and supply signals.

4.5.5 Scalability

Stateless (computed surge lives in the data tier or cache). Scales on request volume and surge computation load. Estimate: **4 instances on average / 10 at peak**, plus periodic batch computation for surge. Responses are cached per cell, which limits calls to the external provider.

4.6 SVC-06 – Payment and Settlement

4.6.1 Responsibility

Manages payment methods (tokenized), authorizes and captures charges through the external provider, handles cash settlement and reconciliation, keeps the driver ledger, schedules payouts and issues invoices. It does **not** calculate fares (SVC-05) and never stores raw card data. *Traceability: FR-08, UC-03.*

4.6.2 APIs

Method	Path	Description
POST / GET	`/v1/payment-methods`	Register or list payment methods.
DELETE	`/v1/payment-methods/{id}`	Remove a payment method.
GET	`/v1/payments/{tripId}`	Payment status for a trip.
GET	`/v1/invoices/{tripId}`	Receipt or invoice.
POST	`/v1/payments/{tripId}/cash-confirmation`	Confirm cash payment (reconciliation).

Method	Path	Description
POST	`/v1/payments/{paymentId}/refund`	Refund (support or admin).
GET	`/v1/drivers/{driverId}/earnings`	Earnings summary.
GET	`/v1/drivers/{driverId}/ledger`	Ledger entries.
POST	`/v1/webhooks/payment-provider`	Callback for asynchronous provider results.

Events published: `PaymentCaptured`, `PaymentFailed`, `PayoutScheduled`, `InvoiceIssued`.
Events consumed: `RideMatched` (pre-authorization), `TripCompleted` (capture), `TripCancelled` (cancellation fees).

4.6.3 Data Owned

`PaymentTransaction`, `DriverLedger`, `Invoice` in a relational database with ACID guarantees. The ledger is append-only (auditable trail).

4.6.4 Dependencies

- **Payment Service Provider** (external): authorization, capture and payouts. Calls carry idempotency keys derived from the trip identifier.
- **Event Bus**: payment workflow driven by trip events (saga, with compensation on failure).

4.6.5 Scalability

Stateless. Scales on event volume and provider call latency, not on telemetry. Estimate: **3 instances on average / 6 at peak**. Concurrency is bounded by provider limits, and a failed payment is retried from a queue without ever blocking trip completion.

4.7 SVC-07 – Feedback and Safety

4.7.1 Responsibility

Records ratings and feedback, maintains aggregate scores, and manages support tickets and safety incidents with their resolution workflow for agents. It does **not** execute refunds or account suspensions; it requests them from SVC-06 and SVC-01. *Traceability: FR-09, FR-10, UC-05.*

4.7.2 APIs

Method	Path	Description
POST	`/v1/trips/{tripId}/ratings`	Submit a rating and optional comment or tip.
GET	`/v1/users/{userId}/rating-summary`	Aggregate rating.
POST	`/v1/incidents`	Report a safety incident.
GET	`/v1/incidents/{id}`	Incident status.
POST	`/v1/support-tickets`	Open a support case (dispute, lost item).
GET	`/v1/support-tickets/{id}`	Ticket status.
PATCH	`/v1/agent/incidents/{id}`	Agent assigns, escalates or resolves.
GET	`/v1/agent/trips/{tripId}/context`	Agent view: trip data gathered from SVC-04.

Events published: `RatingSubmitted`, `RatingAggregateUpdated`, `SafetyIncidentReported`, `SafetyIncidentResolved`, `SupportTicketOpened`. **Events consumed:** `TripCompleted`, `TripCancelled` (open the rating window and prompt for a rating).

4.7.3 Data Owned

`TripRating`, `SupportTicket`, `SafetyIncident` in a document store (variable metadata, free text; attachments in object storage).

4.7.4 Dependencies

- **SVC-04** (synchronous): trip context for agents.
- **SVC-06**: refund requests for disputes (via API).
- **SVC-01**: suspension through `SafetyIncidentReported`.
- **Event Bus**.

4.7.5 Scalability

Stateless. Scales on request volume (post-trip spikes). Estimate: **3 instances on average / 5 at peak**. Rating aggregation is asynchronous and eventually consistent. Safety incident intake is prioritized and never queued behind ratings.

4.8 SVC-08 – Notifications and Communications

4.8.1 Responsibility

Decides what to send, to whom and when; renders templates; respects user preferences; dispatches OTPs, push, SMS and email through the external provider; and logs delivery. It does **not** contain business logic of other domains. *Traceability: supports FR-02 to FR-10 and UC-01 to UC-05.*

4.8.2 APIs

Method	Path	Description
POST	`/v1/internal/notifications/otp`	Internal, synchronous: send an OTP (login critical path).
GET / PUT	`/v1/users/{userId}/notification-preferences`	Read or update channel preferences.
GET	`/v1/users/{userId}/notifications`	Notification history.

Events published: `NotificationDelivered`, `NotificationFailed`. **Events consumed:** `RideMatched`, `DispatchOfferCreated`, `TripStarted`, `TripCompleted`, `TripCancelled`, `PaymentCaptured`, `PaymentFailed`, `InvoiceIssued`, `DriverApproved`, `SafetyIncidentResolved`.

4.8.3 Data Owned

`MessageLog`, `UserPreference` and message templates in a key-value store.

4.8.4 Dependencies

- **Notification / Communications Provider** (external): actual message delivery.
- **Event Bus**: almost all of its input.

4.8.5 Scalability

Stateless worker consumers. Scales on **queue depth** (backlog of events to deliver) and provider throughput. Estimate: **3 instances on average / 8 at peak**. Failures are retried with backoff and dead-lettered. A failure here never affects trips or payments.

5 Data Architecture

5.1 Data Storage Strategy

To ensure independent scalability and fulfill the specific workload requirements of each business domain, the architecture employs polyglot persistence. The storage strategy for each of the eight services is defined as follows:

- **SVC-01 Account and Driver Management**
  - **Storage Type:** Relational Database (RDBMS).
  - **Rationale:** Requires strict ACID compliance for user credentials, profile integrity, and onboarding status.
- **SVC-02 Ride Request Management**
  - **Storage Type:** Document Store (NoSQL).
  - **Rationale:** Provides schema flexibility for varying ride request parameters (e.g., carpool, premium, wheelchair accessible).
- **SVC-03 Matching and Dispatch**
  - **Storage Type:** In-Memory Data Grid / Key-Value Store.
  - **Rationale:** Requires microsecond-level reads and writes to evaluate geospatial algorithms and driver availability in real time.
- **SVC-04 Trip Management and Tracking**
  - **Storage Type:** Wide-Column Store (NoSQL) / Time-Series.
  - **Rationale:** Optimized for high-frequency, write-intensive workloads generated by continuous GPS telemetry updates from drivers.
- **SVC-05 Pricing and Geospatial**
  - **Storage Type:** Relational Database with Spatial Extensions.
  - **Rationale:** Requires complex querying of geofences, regional polygons, and surge multipliers.
- **SVC-06 Payment and Settlement**
  - **Storage Type:** Relational Database (RDBMS).
  - **Rationale:** Financial transactions require strict ACID guarantees to prevent double-spending or lost earnings.

- **SVC-07 Feedback and Safety**
  - **Storage Type:** Document Store (NoSQL).
  - **Rationale:** Safety reports and feedback contain unstructured text and varying metadata.
- **SVC-08 Notifications and Communications**
  - **Storage Type:** Key-Value Store.
  - **Rationale:** Provides fast retrieval for notification templates and delivery logs.

5.2 Data Models

The main entities associated with each service are defined according to the storage strategy and business responsibilities of each domain:

- **SVC-01 Account and Driver Management:** User, DriverProfile, Vehicle, and DocumentVerification.
- **SVC-02 Ride Request Management:** RideRequest, including pickup, drop-off, status, and timestamps.
- **SVC-03 Matching and Dispatch:** ActiveDriverSession, RiderDriverMatch, and DispatchOffer.
- **SVC-04 Trip Management and Tracking:** TripState, TelemetryPing, and RouteHistory.
- **SVC-05 Pricing and Geospatial:** Geofence, PricingRule, and SurgeMultiplier.
- **SVC-06 Payment and Settlement:** PaymentTransaction, DriverLedger, and Invoice.
- **SVC-07 Feedback and Safety:** TripRating, SupportTicket, and SafetyIncident.
- **SVC-08 Notifications and Communications:** MessageLog and UserPreference.

5.3 Data Ownership

Each service has exclusive ownership of its data. To avoid the shared database anti-pattern, no service is permitted to directly query or modify the data store of another service.

Cross-service data access is handled exclusively through synchronous API calls for real-time reads or asynchronous event subscriptions for state replication and event-driven reactions. This approach preserves service autonomy and prevents tight coupling between the persistence layers of different services.

5.4 Consistency

The architecture enforces domain-specific consistency requirements according to the characteristics and criticality of each workload.

- **Strong Consistency:** Applied to SVC-06 Payment and SVC-04 Trip Management for trip creation and completion. Financial ledgers and critical trip state transitions must be immediately consistent to prevent revenue loss or dispatching errors.
- **Eventual Consistency:** Applied to SVC-04 Trip Management for continuous GPS telemetry, SVC-08 Notifications, and SVC-07 Feedback. A slight delay in aggregating a driver's rating or displaying a vehicle's movement on a map is acceptable, prioritizing system availability and scalability.

5.5 Transactions

Local Transactions: ACID guarantees are maintained locally within the boundaries of a single service's database. Each service is responsible for ensuring the atomicity, consistency, isolation, and durability of transactions involving its own data.

Distributed Transactions: Distributed transactions, such as Two-Phase Commit (2PC), are avoided due to their negative impact on availability and scalability. Instead, cross-service operations, such as a trip completion triggering a payment, are managed using the Saga Pattern through asynchronous events.

Services rely on idempotency keys and compensating transactions to handle partial failures across service boundaries, ensuring that distributed workflows can recover without requiring a global transaction coordinator.

5.6 Data Architecture Diagram

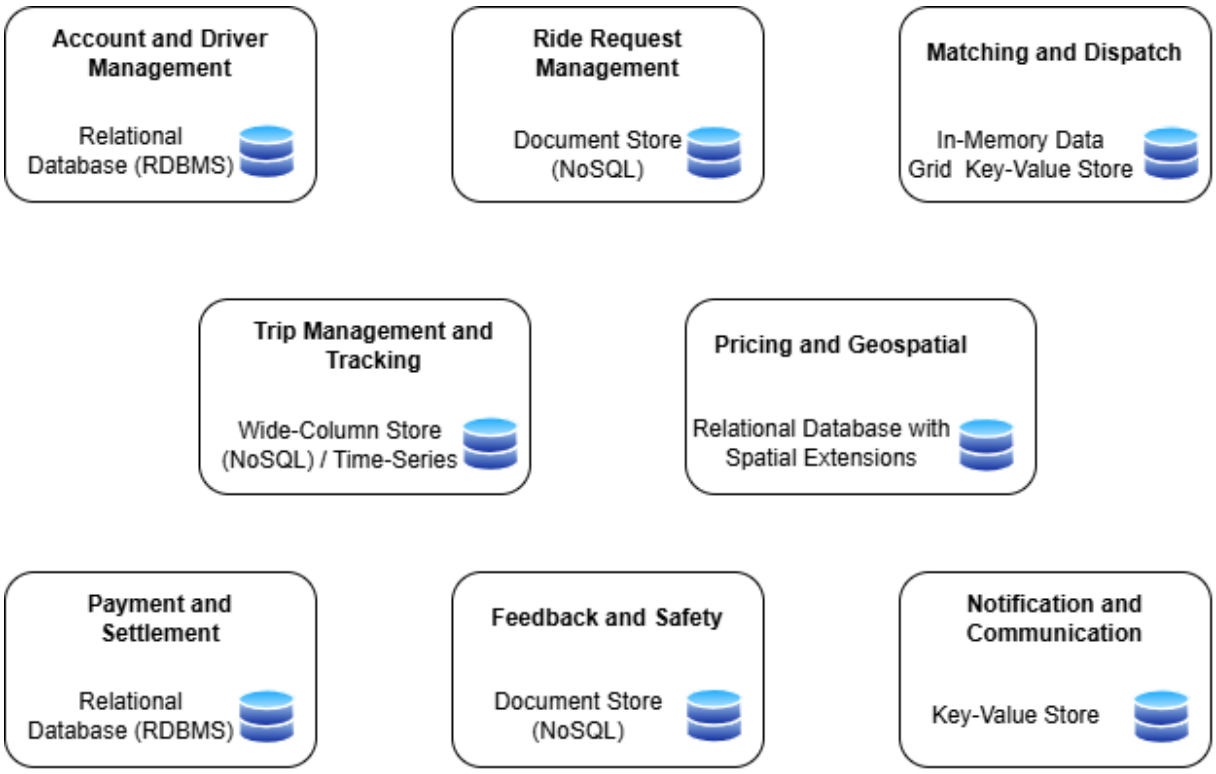


Figure 3: Data ownership and polyglot storage strategy across the eight services.

6 Communication Architecture

6.1 Communication Strategy

Following ADR-003, the system utilizes a hybrid communication strategy. The API Gateway routes external traffic, while internal services communicate using synchronous protocols (REST/gRPC) for operations requiring immediate responses and an asynchronous Event Bus for decoupled, high-volume operations.

This approach allows the architecture to balance low-latency communication for critical interactions with scalability and loose coupling for long-running or event-driven workflows.

6.2 Synchronous Communication

Synchronous communication is used when the caller requires an immediate response, the operation is short-lived, and failures must be surfaced immediately to the user.

The main synchronous interactions are:

- **Caller:** API Gateway → **Callee:** SVC-02 Ride Request Management.

  - **Operation:** `POST /request`
  - **Latency:** < 500 ms

- **Caller:** SVC-02 Ride Request Management $\rightarrow$ **Callee:** SVC-05 Pricing and Geospatial.
  - **Operation:** `GET /estimate`
  - **Latency:** < 100 ms
  - **Purpose:** Provides an immediate fare calculation before displaying the estimated price to the rider.

6.3 Asynchronous Communication

Asynchronous communication is used for long-running processes, high-frequency updates, and interactions where producers and consumers should remain decoupled.

The Event Bus uses an at-least-once delivery guarantee. Consequently, all consumer services must implement idempotent operations to safely handle potential duplicate event deliveries.

The main asynchronous interactions are:

- **Producer:** SVC-04 Trip Management and Tracking.
  - **Event:** `DriverLocationUpdated`
  - **Consumer:** SVC-03 Matching and Dispatch
  - **Delivery:** High-throughput topic used to support real-time matching algorithms.

- **Producer:** SVC-04 Trip Management and Tracking.
  - **Event:** `TripCompleted`
  - **Consumers:** SVC-06 Payment and Settlement and SVC-07 Feedback and Safety
  - **Purpose:** SVC-06 processes the rider's payment, while SVC-07 uses the event to prompt the rider for a trip rating.

6.4 Communication Diagram

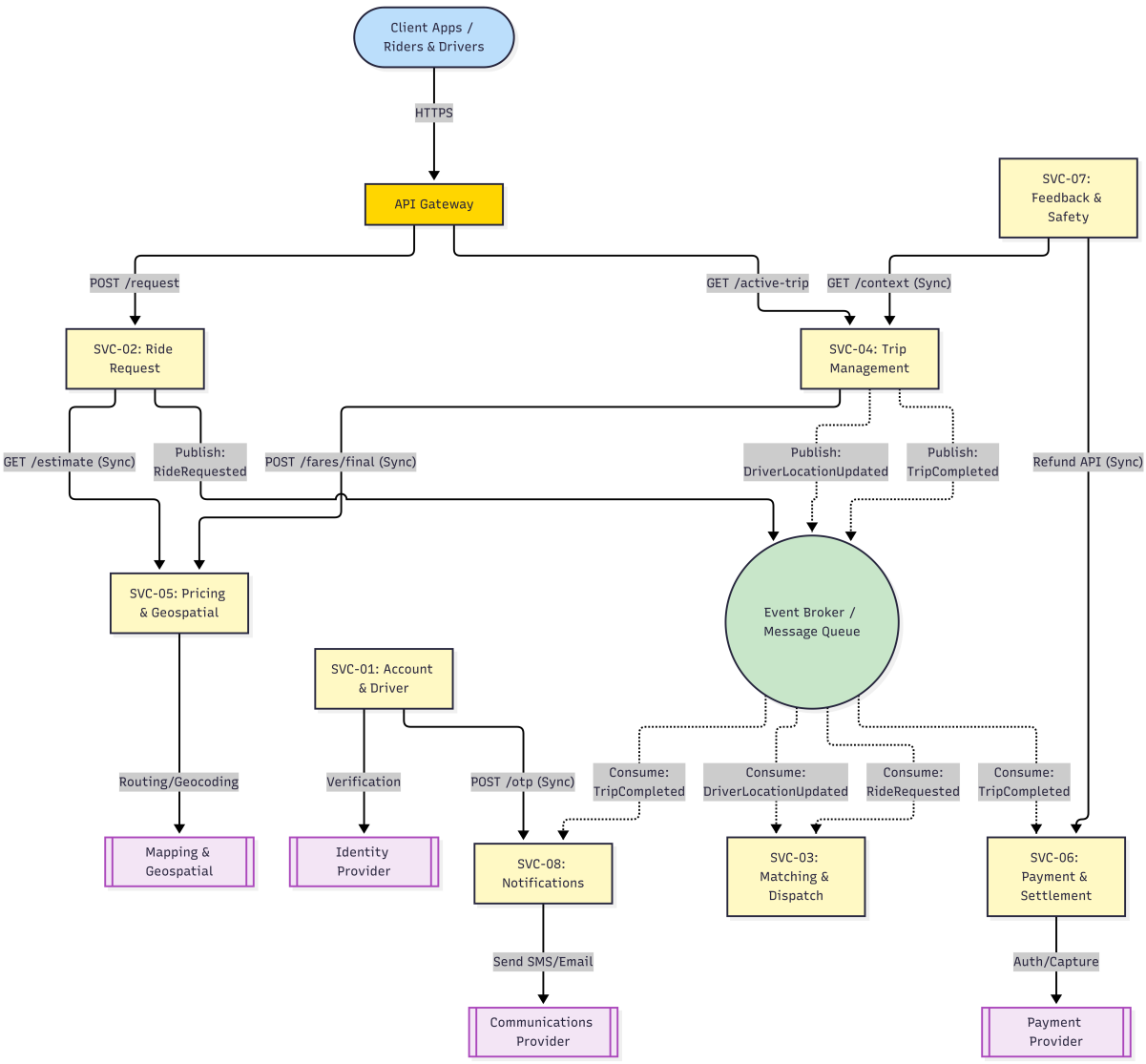


Figure 4: Synchronous and asynchronous interactions between services.

6.5 Communication Justification

The hybrid communication strategy is used to balance responsiveness, reliability, scalability, and service decoupling.

If all communication were synchronous, a failure in services such as Notifications or Feedback could cause the Trip Completion process to fail, potentially creating cascading outages across the system. By using asynchronous events for workflows such as `TripCompleted`, SVC-04 Trip Management and Tracking can finalize the ride without waiting for downstream services to complete their processing. Payment and feedback-related operations can then be processed reliably in the background.

Synchronous communication is therefore reserved for interactions that belong to the critical path of the rider's request, such as submitting a ride request and obtaining an im-

mediate fare estimate. Asynchronous communication is preferred for event-driven workflows, high-frequency updates, and operations where immediate responses are not required.

7 Architecture Decision Records

This section documents the main architectural decisions that define the software architecture of the ride-sharing platform. The decisions are based on the functional requirements, non-functional requirements, and workload characteristics defined in Delivery 1.

7.1 ADR-001 – Service Decomposition

7.1.1 Context

The platform must support ride requests, driver matching, trip management, payments, feedback, safety, and driver onboarding. These capabilities have different workload and scalability characteristics. Real-time matching and location tracking are particularly latency-sensitive, while payment and settlement require stronger consistency.

7.1.2 Decision

The platform will be decomposed into independently deployable services based on distinct business responsibilities.

The architecture will separate at least the following responsibilities:

- Driver and account management.
- Ride request management.
- Matching and dispatch.
- Trip management and tracking.
- Pricing and geospatial services.
- Payment and settlement.
- Feedback and safety.
- Notifications and communications.

7.1.3 Alternatives

- Monolithic architecture.
- Modular monolith.
- Independently deployable services.

7.1.4 Rationale

Independent services allow high-load capabilities such as matching, tracking, and ride requests to scale without requiring the entire platform to scale together. They also provide clear boundaries between capabilities with different consistency and availability requirements.

7.1.5 Trade-offs and Consequences

This decision improves independent scalability and service isolation, but introduces additional communication, monitoring, failure-handling, and consistency complexity.

Services must therefore communicate through explicit interfaces and must not depend on direct access to another service's internal implementation or data.

7.2 ADR-002 – Service Data Ownership

7.2.1 Context

The platform manages different types of data, including driver information, ride requests, trip state, payments, ratings, and support cases. These data have different consistency and access requirements.

7.2.2 Decision

Each service will own the data associated with its business responsibility. Other services will access that information through defined service interfaces or asynchronous events rather than directly accessing another service's data store.

7.2.3 Alternatives

- A shared database for all services.
- A centralized data-access service.
- Independent data ownership per service.

7.2.4 Rationale

Independent data ownership reduces coupling and allows each service to evolve and scale according to its workload. It also prevents changes in one business domain from directly affecting the internal data structures of another service.

7.2.5 Trade-offs and Consequences

Independent data ownership makes cross-service queries and transactions more complex. Some information may need to be exchanged through APIs or events, and some views may eventually require replicated or derived data.

No service should directly modify data owned by another service.

7.3 ADR-003 – Hybrid Communication Strategy

7.3.1 Context

The platform contains interactions that require an immediate response, such as requesting a ride or obtaining an estimated fare. It also contains event-driven operations such as notifications, location updates, and other processes that do not always require the initiating service to wait for completion.

The workload model also identifies a high-frequency, write-intensive stream of driver location updates.

7.3.2 Decision

The architecture will use a combination of synchronous and asynchronous communication.

Synchronous communication will be used when the caller requires an immediate response or confirmation.

Asynchronous communication will be used for event-driven operations, high-frequency updates, notifications, and operations that can be processed independently.

7.3.3 Alternatives

- Fully synchronous communication.
- Fully asynchronous communication.
- Hybrid synchronous and asynchronous communication.

7.3.4 Rationale

A hybrid approach allows latency-sensitive operations to obtain immediate responses while reducing coupling for high-volume or event-driven operations.

This is particularly important for driver location updates and other high-frequency events that could otherwise create unnecessary synchronous dependencies between services.

7.3.5 Trade-offs and Consequences

Synchronous communication is simpler for immediate interactions but increases runtime coupling between services.

Asynchronous communication improves decoupling and scalability but introduces concerns related to delivery, ordering, retries, and eventual consistency.

The communication pattern must therefore be selected according to the characteristics of each interaction.

7.4 ADR-004 – Consistency and Transaction Management

7.4.1 Context

Not all platform data requires the same consistency guarantees. Delivery 1 identifies trip state, payment processing, driver earnings, and settlement as operations requiring stronger correctness, while location information can tolerate some degree of staleness.

7.4.2 Decision

The architecture will use different consistency strategies according to the business importance of each data domain.

Strong consistency will be prioritized for critical state transitions and financial operations.

Eventual consistency will be acceptable for information such as location updates, notifications, and derived information where temporary staleness does not compromise the correctness of the transaction.

Cross-service business processes will be coordinated through explicit service interactions rather than relying on a shared transaction across all services.

7.4.3 Alternatives

- Strong consistency for all data.
- Eventual consistency for all data.
- Consistency according to the business domain.

7.4.4 Rationale

Applying the same consistency model to every operation would either introduce unnecessary coordination or weaken guarantees for critical operations.

A domain-specific approach allows financial and trip-critical data to maintain stronger correctness while allowing high-frequency information to prioritize availability and responsiveness.

7.4.5 Trade-offs and Consequences

The architecture becomes more complex because services must explicitly define their consistency requirements.

Operations involving multiple services may require coordination, retries, compensation, or reconciliation mechanisms.

7.5 ADR-005 – Independent Scalability and Geographic Distribution

7.5.1 Context

The workload model defines a global platform with approximately 250,000 average concurrent users, more than 500,000 peak concurrent users, 45 million average requests per hour, and up to 120 million requests per hour during peak periods. The platform must also support more than 100,000 simultaneous active rides.

The workload includes geographically concentrated demand and scenarios involving regional failures.

7.5.2 Decision

Services will be designed to scale independently according to their workload characteristics.

The architecture will support geographic distribution of system capabilities so that traffic and processing can be handled close to the regions where demand occurs.

Critical services will be designed with redundancy so that a failure in one geographic area does not necessarily interrupt the complete platform.

7.5.3 Alternatives

- Centralized processing for all geographic regions.
- Independent regional deployments with no coordination.
- Distributed architecture with coordinated regional operation.

7.5.4 Rationale

The workload is global and highly variable. Matching, trip tracking, and other real-time capabilities may experience concentrated demand in specific geographic areas.

Independent scaling and geographic distribution allow the architecture to handle regional demand variations while supporting the availability target defined in Delivery 1.

7.5.5 Trade-offs and Consequences

Geographic distribution increases architectural complexity and requires mechanisms for coordinating state across regions.

The architecture must also define how requests are routed, how regional failures are handled, and how critical information remains consistent across geographic boundaries.

8 Requirements Traceability

This section establishes the relationship between the requirements defined in Delivery 1 and the software architecture proposed in Delivery 2.

8.1 Functional Requirements Traceability

Each functional requirement is mapped to the service or services primarily responsible for supporting the corresponding actor action.

ID	Requirement	Responsible Service(s)	Use Case
FR-01	Driver onboarding	SVC-01 Account and Driver Management	UC-04
FR-02	Driver availability and ride assignment	SVC-01 Account and Driver Management; SVC-03 Matching and Dispatch	UC-02
FR-03	Ride request creation	SVC-02 Ride Request Management	UC-01
FR-04	Ride assignment acceptance or rejection	SVC-03 Matching and Dispatch	UC-01
FR-05	Fare and arrival-time review	SVC-05 Pricing and Geospatial	UC-01
FR-06	Pickup confirmation and trip status updates	SVC-04 Trip Management and Tracking	UC-01, UC-02

ID	Requirement	Responsible Service(s)	Use Case
FR-07	Real-time trip location and progress	SVC-04 Trip Management and Tracking; SVC-05 Pricing and Geospatial	UC-02
FR-08	Payment method and payment information	SVC-06 Payment and Settlement	UC-03
FR-09	Trip rating and feedback	SVC-07 Feedback and Safety	UC-05
FR-10	Safety incident and support reporting	SVC-07 Feedback and Safety	UC-05

Table 13: Functional requirements traceability

8.2 Non-Functional Requirements Traceability

The non-functional requirements are addressed through multiple architectural elements rather than by a single service.

ID	Requirement	Architectural Elements
NFR-01	Concurrent users	Independent service scaling and distributed service deployment.
NFR-02	Average request rate	Independent service scaling, asynchronous communication, and separation of high-volume workloads.
NFR-03	Peak request rate	Independent scaling, workload isolation, and distributed processing.
NFR-04	Peak load duration	Horizontal scalability and sustained processing capacity.
NFR-05	Response time	Low-latency synchronous interactions, service decomposition, and geographically distributed processing.
NFR-06	99.99% availability	Service redundancy, geographic distribution, failure isolation, and asynchronous decoupling.
NFR-07	20 TB initial data volume	Independent data ownership and service-specific data storage.
NFR-08	15 TB/year data growth	Independent data scaling and separation of data according to business domain.

ID	Requirement	Architectural Elements
NFR-09	Global geographic scope	Geographic distribution and independent regional service operation.

Table 14: Non-functional requirements traceability

8.3 Architecture Decision Traceability

The main architecture decisions are directly motivated by the requirements and workload characteristics defined in Delivery 1.

ADR	Decision	Driving Requirements
ADR-001	Service decomposition	FR-01–FR-10; NFR-01–NFR-06
ADR-002	Service data ownership	FR-06, FR-08, FR-09, FR-10; NFR-07, NFR-08
ADR-003	Hybrid communication strategy	FR-02–FR-07; NFR-02, NFR-03, NFR-05
ADR-004	Consistency and transaction management	FR-06, FR-08, FR-10; NFR-05, NFR-06
ADR-005	Independent scalability and geographic distribution	NFR-01–NFR-06, NFR-09

Table 15: Architecture decision traceability

8.4 Traceability Summary

The architecture is derived from the functional responsibilities and operational characteristics established in Delivery 1. Each functional requirement is assigned to one or more services, while non-functional requirements are addressed through architectural properties such as independent scalability, data ownership, communication strategies, redundancy, and geographic distribution.

The traceability also ensures that the main architecture decisions are supported by previously identified requirements rather than being introduced independently of the system needs.